

به نام الهام بخش دانا

راهنمای یادگیری فریم ورک

Vue.js

(آشنایی اولیه - نقشه یادگیری - کارآمدترین متدها برای استفاده - منابعی برای آموزش)

حامد عزیزی پناه

پاییز ۱۴۰۲

فهرست مطالب

۳	پیش‌گفتار
۴	تاریخچه پیدایش Vue.js
۵	Vue.JS چیست؟
۶	Vue یک کتابخانه است یا فریمورک؟
۷	پیش‌نیازهای یادگیری Vue.js
۸	نقشه راه یادگیری (Roadmap)
۱۱	نمونه کدها
۱۴	منابع پیشنهادی برای یادگیری

از زمانی که فریمورک‌های مختلفی برای جاوااسکریپت منتشر می‌شود بسیاری از توسعه‌دهندگان همواره به این فکر می‌کردند که برای توسعه لایه فرانت‌اند می‌شود انتخابی همه‌جانبه‌تر و در عین حال ساده‌تر وجود داشته باشد. این موضوع تا زمانی که ویوجی‌اس توسعه یافت ادامه یافت و با انتشار Vue.JS همه مشکلات حل گشت.

ویوجی‌اس یک فریمورک MVV یا Model-View-Viewmodel جاوااسکریپتی است که به شما کمک می‌کند تا رابط‌های کاربری اپلیکیشن‌ها را توسعه دهید و همچنین در فرایند ساخت اپلیکیشن‌های تک صفحه‌ای یا Single Page Application سرعت و دقت عمل بیشتری داشته باشید.

ایده اصلی ایجاد ویوجی‌اس آن بود که بتواند تمام ویژگی‌های کاربردی انگولار را در یک پکیج سبک‌تر وارد کرده و از پیچیدگی‌های توسعه آن بکاهد. باید بگوییم که در سال‌های اخیر به خوبی اثبات شده که این پروژه واقعا موفقیت‌آمیز بوده است. در حال حاضر برای یادگیری فریمورک‌های دیگر جاوااسکریپت مانند انگولار و ری‌اکت شما به یادگیری موضوعات جانبی مانند تایپ‌اسکریپت، JSX و... نیاز دارید اما از آنجایی که ویوجی‌اس سعی داشته تا حد ممکن ساده باشد، شما در فرایند یادگیری آن تنها نیاز دارید که دانشی در رابطه با HTML/CSS/JS داشته باشید. در حال حاضر شرکت‌های بسیار زیادی برای توسعه لایه فرانت‌اند خود از ویوجی‌اس استفاده می‌کنند.

در این مقاله ما برآنیم که پیش‌نیازهای لازم و همچنین نقشه راه یادگیری این فریمورک را بر اساس گرایش تیم توسعه خود مشخص کنیم. در انتها هم منابع مفیدی برای یادگیری معرفی خواهیم کرد.

تاریخچه پیدایش Vue.js

Vue.js، که امروزه به عنوان یکی از پرکاربردترین فریمورک‌های جاوااسکریپت شناخته می‌شود، ریشه در تجربیات و دیدگاه‌های **Evan You** دارد. وی در زمان کار خود با Angular.js در گوگل، به این نتیجه رسید که اگرچه Angular.js یک فریمورک قدرتمند است، اما می‌تواند به شکلی ساده‌تر و انعطاف‌پذیرتر بازطراحی شود. در نتیجه، در سال ۲۰۱۴، او کار بر روی یک پروژه شخصی را آغاز کرد که در نهایت به تولد Vue.js انجامید. هدف او از این پروژه، ایجاد یک فریمورک سبک وزن بود که بدون نیاز به ابزارهای اضافی، توسعه‌دهندگان را قادر سازد تا رابط‌های کاربری دینامیک و واکنش‌گرا بسازند.

Vue.js در مقایسه با سایر فریمورک‌های مشابه آن زمان، مثل Angular یا React، با پیچیدگی کمتری همراه بود. این فریمورک با تاکید بر مفاهیم اصلی و اساسی جاوااسکریپت و دیگر فناوری‌های وب استاندارد مثل HTML و CSS، به توسعه‌دهندگان اجازه می‌داد تا بدون نیاز به یادگیری ابزارهای تخصصی جدید، پروژه‌های خود را سریع‌تر و کارآمدتر پیش ببرند. Vue.js با ارائه مستندات فراگیر و کامل و جامعه‌ای فعال و پشتیبانی‌کننده، به سرعت محبوبیت پیدا کرد و توانست جایگاه خود را در میان توسعه‌دهندگان و شرکت‌ها به عنوان یک انتخاب مطمئن برای توسعه فرانت‌اند تثبیت کند.

از زمان انتشار اولیه‌اش، Vue.js به طور مستمر در حال بهبود و تکامل بوده است، با اضافه شدن قابلیت‌های جدید، بهینه‌سازی‌های عملکردی و توسعه اکوسیستمی که ابزارهای توسعه، افزونه‌ها و قالب‌های پیش‌ساخته را شامل می‌شود. این پیشرفت‌ها همراه با حفظ هسته‌ی اصلی ساده و سبک فریمورک بوده‌اند که به توسعه‌دهندگان اجازه می‌دهد تا با یادگیری اندکی از Vue.js بتوانند پروژه‌های بزرگ و پیچیده‌ای را مدیریت کنند.

Vue.JS چیست؟

مطابق با مستندات موجود در وبسایت [Vuejs.org](https://vuejs.org) فریمورک ویوجی‌اس یک فریمورک پیش‌رونده برای ایجاد رابط کاربری اپلیکیشن‌ها و ایجاد اپلیکیشن‌های تک صفحه‌ای‌ست. در ویوجی‌اس می‌توانید تقریباً تمام امکانات موجود در فریمورک‌های ری‌اکت و انگولار را البته با روشی ساده‌تر برای توسعه مشاهده کنید. این فریمورک بصورت تخصصی تنها روی لایه View خود تمرکز داشته و این موضوع باعث می‌شود که ادغام شدن با دیگر کتابخانه و ابزارها بسیار ساده‌تر شود. در حال حاضر از ویوجی‌اس در پروژه‌های ساده و پایه تا پروژه‌های پیشرفته و پیچیده استفاده می‌شود و این امر به لطف ماژولار بودن ویوجی‌اس است. کتابخانه‌ها و ابزارهای توسعه کمی بسیاری برای ویوجی‌اس ایجاد شده که این موضوع دامنه گستردگی آن را بی‌پایان می‌کند.

وجود توسعه‌دهندگان و انجمن‌های برنامه‌نویسی فعال در زمینه ویوجی‌اس به شما کمک شایانی در رفع مشکلات احتمالی کار با آن خواهد کرد. از طرفی دیگر ویوجی‌اس با بروزرسانی‌های اخیری که داشته به شما قابلیت توسعه اپلیکیشن‌های موبایلی را نیز می‌دهد. البته تکامل این مورد نیاز به زمان و کارهای بیشتری دارد.

Vue.js با تکیه بر ماهیت تطبیق‌پذیری و منعطف خود، به‌صورت مستمر در حال به‌روزرسانی و ارتقاء است تا با نیازهای تغییرپذیر توسعه‌دهندگان و انتظارات مداوم بازار همگام باشد. این پویایی نه تنها از طریق اصلاحات منظم در هسته اصلی فریمورک تجلی می‌یابد بلکه همچنین از طریق گسترش اکوسیستم اطراف آن، با افزودنی‌ها، پلاگین‌ها و ابزارهای متنوع که به صورت مشارکتی توسط جامعه توسعه داده شده‌اند، تقویت می‌شود. این امکانات اضافی، ویوجی‌اس را نه تنها به یک گزینه ایده‌آل برای ساخت صفحات وب ساده تبدیل می‌کند بلکه قدرت و انعطاف لازم برای پشتیبانی از اپلیکیشن‌های سنگین و مقیاس بزرگ را نیز فراهم می‌آورد.

Vue یک کتابخانه است یا فریمورک ؟

برای درک عمیق‌تر از ماهیت Vue.js و تشخیص اینکه آیا به عنوان یک کتابخانه یا یک فریمورک عمل می‌کند، باید ابتدا تعاریف دقیق و کاربردی از این دو اصطلاح را مورد بررسی قرار دهیم:

کتابخانه (Library): در زمینه برنامه‌نویسی، کتابخانه به مجموعه‌ای از کدهای قابل استفاده مجدد گفته می‌شود که برای سهولت بخشیدن به توسعه نرم‌افزار، وظایف مشخصی را انجام می‌دهند. شما می‌توانید در هر بخشی از برنامه خود که لازم دیدید، از این کدها استفاده کنید و به انتخاب خود کنترل جریان کار را در دست بگیرید.

فریمورک (Framework): فریمورک، ساختار یا قالبی پیش‌فرض است که برای توسعه و سازماندهی کد نرم‌افزار به کار می‌رود و معمولاً مجموعه‌ای از کتابخانه‌ها را در خود جای داده است. تفاوت اصلی آن با کتابخانه در این است که فریمورک خود اسکلت کلی جریان کاری را تعیین می‌کند و توسعه‌دهندگان را به دنبال کردن قوانین و الگوهای مشخصی هدایت می‌کند.

با این اوصاف، Vue.js را می‌توان در دو بعد مورد بررسی قرار داد: به صورت مستقل، Vue.js به عنوان یک کتابخانه فعالیت می‌کند که برای ساخت و دستکاری لایه‌ی مشاهده (View) در اپلیکیشن‌های وب به کار می‌رود. اما زمانی که Vue با افزونه‌ها و ابزارهای تکمیلی مانند Vue Router برای مسیریابی و Vuex برای مدیریت حالت، همراه می‌شود، به یک فریمورک کامل تبدیل می‌گردد که قادر است کلیت یک وبسایت پیچیده و کامل SPA را پشتیبانی کند.

این گستردگی و انعطاف‌پذیری Vue است که به توسعه‌دهندگان این قدرت را می‌دهد تا به تناسب نیاز پروژه خود، تصمیم بگیرند که آیا Vue را به عنوان بخشی از معماری وسیع‌تر خود استفاده کنند یا به صورت یک فریمورک مستقل از آن بهره ببرند. این مسئله، Vue را به ابزاری بسیار مناسب برای انواع پروژه‌ها، از سایت‌های ساده تا اپلیکیشن‌های مختلف تبدیل می‌کند.

پیش‌نیازهای یادگیری Vue.js

۱: دانش HTML و CSS

- آشنایی با نحوه‌ی ساختاردهی محتوا با استفاده از HTML
- درک مفاهیم پایه‌ای CSS و توانایی استایل‌دهی به المان‌ها

۲: جاوا اسکریپت و ES۶

- تسلط بر مبانی جاوا اسکریپت، مانند متغیرها، توابع، و اشیاء
- آشنایی با ویژگی‌های ES۶، مانند `let`، `const`، `arrow functions`، و `template literals`

۳: مفاهیم پایه‌ای برنامه‌نویسی

- درک مفهوم DOM و چگونگی تعامل با آن
- مفاهیم اولیه مانند حلقه‌ها، شرط‌ها، و داده‌ساختارها

۴: ابزارهای توسعه وب

- آشنایی با `npm` و `yarn` برای مدیریت بسته‌ها
- درک اولیه‌ای از سیستم‌های کنترل نسخه، مانند `Git`

۵: فهم responsive design

اهمیت طراحی واکنش‌گرا و تکنیک‌های اصلی برای ایجاد رابط‌های کاربری که بر روی انواع دستگاه‌ها به خوبی نمایش داده می‌شوند.

نقشه راه یادگیری (Roadmap)

در اینجا به نقشه راه یادگیری ۳ Vue.js مطابق عملکرد حال حاضر تیم ما می‌پردازیم:

۱- مقدمات: Vue.js

آشنایی با اصول اولیه Vue.js شامل داده‌های واکنش‌گرا، مؤلفه‌ها و دستورالعمل‌ها.
درک چرخه زندگی مؤلفه‌ها و استفاده از توابع چرخه زندگی در Composition API

۲- یادگیری ES۶+ و JavaScript

تسلط بر مفاهیم پیشرفته JavaScript مانند کلاس‌ها، ماژول‌ها، توابع فلش (arrow functions) و سایر ویژگی‌های ES۶+.

۳- متدهای Composition API

یادگیری و به کارگیری ref, reactive, computed, watch و toRefs برای مدیریت state و واکنش‌گرایی.

۴- فرمت: <script setup>

تمرین نوشتن مؤلفه‌های تک‌فایلی با استفاده از فرمت <script setup> برای سادگی و کارآمدی بیشتر.

۵- اعتبارسنجی با پکیج vee-validate

تعریف قوانین اعتبارسنجی: استفاده از vee-validate برای تعریف قوانین اعتبارسنجی انعطاف‌پذیری: سازگاری با انواع فرم‌ها و ورودی‌ها، امکان اعتبارسنجی گروهی و تکی.

۶- مسیره‌ی ها با Vue Router

اجرای مسیریابی و ناوبری درون برنامه‌ای با Vue Router.

۷- State Management

آشنایی با مدیریت حالت فراگیر با Pinia و انتقال حالت‌ها و لاجیک‌های برنامه‌ای.

۸- کتابخانه‌ها: Element-Plus - ۳ Vuetify

یادگیری و به کارگیری این کتابخانه‌های UI برای طراحی رابط کاربری و اجزای آن.

۹- مدیریت پکیج با npm و yarn

درک چگونگی نصب و مدیریت وابستگی‌ها با npm و yarn

۱۰- کامپایلر Vite

یادگیری استفاده از Vite به عنوان کامپایلر و ابزار توسعه برای بهبود تجربه توسعه و بهینه‌سازی برنامه.

۱۱- بهترین شیوه‌ها و الگوهای طراحی:

مطالعه و به کارگیری الگوهای طراحی مؤثر و بهترین شیوه‌ها برای کدنویسی پایدار و قابل نگهداری.

۱۲- کار با API ها:

تمرین برقراری ارتباط با سرورها و API ها و مدیریت وضعیت‌های آسنکرون.

نمونه کدها

در ادامه نمونه کدهای مورد استفاده تیم ما در موضوع Vue.js آمده است.

کد زیر با متد **Composition Api** و با فرمت **script setup** و با استفاده از کتابخانه **Vuetify** نسخه ۳ نوشته شده است.

یک کامپوننت با بهره گیری از متغیرهای reactive:

```
<script setup>
import { defineProps, reactive, computed, watch } from "vue";
import {
  filterNumbers,
  filterNumbersWithSep,
} from "@/services/tools/filter-numbers";

const props = defineProps({
  editField: Object,
  propField: Object,
});

const state = reactive({
  rawNumberValue: props.editField.value,
});

const numberValue = computed({
  get: () => {
    const filteredNumber = filterNumbers(state.rawNumberValue,
props.propField.filterOptions.zeroAllowed);
```

```

        return props.propField.filterOptions.separator
            ? filterNumbersWithSep(filteredNumber,
props.propField.filterOptions.zeroAllowed)
            : filteredNumber;
    },
    set: (value) => {
        const filteredNumber = filterNumbers(value,
props.propField.filterOptions.zeroAllowed);
        if (state.rawNumberValue !== filteredNumber) {
            state.rawNumberValue = filteredNumber;
            props.editField.value = filteredNumber;
        }
    }
});

watch(() => props.editField.value, (newVal) => {
    if (newVal !== state.rawNumberValue) {
        state.rawNumberValue = newVal;
    }
});
</script>

<template>
    <div class="flex">
        <v-text-field
            class="field"
            variant="solo"
            density="compact"
            :placeholder="propField.filterOptions?.placeholder"
            v-model="numberValue"
            :style="propField.filterOptions?.styles"
            hide-details

```

```
></v-text-field>
<span v-if="propField.suffix" class="suffix">
  {{ propField.suffix }}
</span>
</div>
</template>

<style scoped>
.flex {
  align-items: center;
  gap: 10px;
}
.field {
  width: 160px;
}
</style>
```

منابع پیشنهادی برای یادگیری

در ادامه منابع پیشنهادی ما برای یادگیری Vue.js آمده است:

- [آموزش کامل vue.js \(ویو جی اس\) - پروژه محور و اصولی](#)
- [دوره ۴۰ ساعته آموزش VueJS همراه با مدرک پایان دوره \(با تأیید فنی و حرفه‌ای\)](#)